Lecture 18 — Mid-Year Review: Problem-Solving Method and Mock Contest

> Year Curriculum · Prerequisites: Weeks 1–17 · Estimated time: 1 hour review + 3-hour mock contest + 1 hour debrief

1. Overview

Seventeen weeks in, you own a substantial toolbox: complexity analysis and brute force, sorting and greedy, binary search, two pointers, prefix sums, backtracking, bitmasks, number theory, stacks and queues, heaps and ordered sets, dynamic programming, BFS/DFS and shortest paths. The difficulty in contests is rarely "I have never seen this algorithm" — it is "I did not realise this problem was *that* algorithm", or "I knew the idea but my code had a bug".

This week has three parts:

1. **A problem-solving method** you can run on any unseen problem, including a table that converts constraints into a target complexity and a table of *signals* that point to techniques.
2. **A 3-hour mock contest** of eight real problems spanning the first half of the year.
3. **A debrief** of your own contest log, plus an upsolving plan. The solutions are held by the coach and are not part of these notes.

If you are reading these notes before sitting the mock contest, stop after Section 4 and do the contest first. The debrief is far more valuable once you have struggled with the problems.

2. The topic map

Weeks	Block	What you should be able to write from memory
1–3	Foundations	fast I/O, reading input formats, estimating operation counts, complete search over subsets/permutations
4	Sorting & greedy	custom comparators, exchange arguments
5	Binary search	`lower_bound`/`upper_bound`, binary search on a monotone predicate
6	Two pointers	pair sums in sorted arrays, longest window with a property
7	Prefix sums	1D/2D prefix sums, difference arrays
8	Backtracking	subsets, permutations, N-Queens with pruning
9	Bits	masks as subsets, `__builtin_popcount`, iterating over submasks

Weeks	Block	What you should be able to write from memory
10–11	Number theory	sieve, factorisation, gcd, fast power, modular inverse, nCr mod p
12	Stacks & queues	monotonic stack (next smaller element), monotonic deque (window minimum)
13	Heaps, sets, maps	`priority_queue`, `set`/`multiset` with `lower_bound`, `map` counting
14–15	DP	state/transition/base/order/answer; knapsack, LIS, LCS, edit distance, grid DP
16	Graphs I	adjacency lists, BFS, DFS, components, flood fill, bipartite check
17	Graphs II	Dijkstra, Bellman-Ford, Floyd-Warshall, 0-1 BFS, state graphs

Be honest with yourself: mark each row green (I can code it cold), yellow (I need to look things up), or red (I don't really understand it). The second half of the year builds directly on these — trees (Week 20) need DFS; segment trees (Week 23) need recursion and prefix-sum thinking; bitmask DP (Week 26) needs both Weeks 9 and 15.

3. A problem-solving method

3.1 Understand the problem

1. **Read the entire statement**, including input/output format, constraints and notes. Constraints are part of the problem, not decoration.
2. **Restate it** in one precise sentence without the story. "Given an array, count pairs (l, r) with sum of a[l..r] divisible by n." If you cannot, you have not understood it yet.
3. **Work through the samples by hand.** If your understanding gives a different output, find out why *now*, not after three wrong submissions.
4. **Invent small cases**, including edge cases: n = 1, all elements equal, all negative, the maximum values, an empty answer, a disconnected graph, a start cell that is blocked.

3.2 From constraints to complexity

A C++ program performs roughly 10^8 simple operations per second (more for tight array loops, fewer for `map` or heavy recursion). Combine that with the time limit to get a budget:

Largest n	Feasible complexity	Typical techniques
≤ 10–12	O(n!)	permutations, backtracking
≤ 20–25	$O(2^n \cdot n)$	bitmask enumeration; meet in the middle up to ~40
≤ 500	$O(n^3)$	Floyd-Warshall, interval DP
≤ 5000	$O(n^2)$	2D DP, all pairs

Largest n	Feasible complexity	Typical techniques
$\le 2 \cdot 10^5$ – 10^6	O(n log n) or O(n)	sorting, binary search, heaps, BFS, prefix sums, two pointers
$\le 10^{18}$	O(log n) or O(√n)	math, binary search on the answer, fast exponentiation

Reading this table backwards is just as useful: if n ≤ 20, the setter is practically telling you to enumerate subsets; if n ≤ 500, a cubic algorithm is probably intended.

3.3 Find a solution

5. **Start with brute force.** Describe the most naive correct algorithm and its complexity. Even if it is too slow, it clarifies the problem, and you can later use it to test your fast solution.
6. **Ask what makes brute force slow** and look for structure: Is some predicate monotone (binary search)? Does order not matter (sort first)? Do subproblems repeat (DP)? Are there objects and relations between them (graph)? Can we count the complement instead? Can we reverse time or reverse the edges?
7. **Match signals** to techniques (Section 3.4).
8. **Convince yourself it is correct** — a proof sketch for greedy, a clear state definition for DP — and run it on your small cases on paper.
9. **Code it cleanly.** Name variables meaningfully, avoid clever one-liners under pressure, and use `long long` by default when values can exceed $2 \cdot 10^9$.
10. **Test** on the samples and your own cases before submitting.

If you are stuck for 15 minutes: reread the statement (really), solve tiny instances completely by hand looking for a pattern, or switch to another problem and come back.

3.4 Signals

Statement says…	Consider…
"minimise the maximum", "smallest T such that…"	binary search on the answer
sorted data, pairs/triples with a target sum	two pointers
many range-sum queries on a static array	prefix sums
many range increments, read at the end	difference array
"nearest smaller/greater element", histogram areas	monotonic stack
min/max of every window of length k	monotonic deque
repeatedly take the cheapest/largest item	priority queue
"count the number of ways", "minimum cost", overlapping choices	DP
subset under a weight/cost limit	knapsack
two strings, align or compare	prefix × prefix DP

Statement says…	Consider…
$n \le 20$, choose a subset	bitmasks
answer modulo 10^9+7, binomial coefficients	modular arithmetic, precomputed factorials
maze, grid, "minimum number of moves"	BFS
roads with lengths, "cheapest route"	Dijkstra
split into two groups with constraints between pairs	bipartite check

Signals are hints, not rules. Their main use is to generate candidate ideas quickly; you still have to check that the idea fits the constraints.

4. The mock contest

Format. 3 hours, individual, ICPC scoring (number solved, then total time with a 20-minute penalty per rejected submission on problems eventually solved). Use the CSES, Codeforces and AtCoder judges directly, or set up a Codeforces Gym / vjudge contest with these problems. Paper and your own notes are allowed; searching for solutions is not.

Keep a log. For each problem write when you opened it, when you had an idea, when you submitted, and the reason for every rejection. You will use this log in the debrief.

#	Problem	Link
A	Codeforces 1352C — K-th Not Divisible by n	https://codeforces.com/problemset/problem/1352/C
B	CSES 1640 — Sum of Two Values	https://cses.fi/problemset/task/1640
C	CSES 1620 — Factory Machines	https://cses.fi/problemset/task/1620
D	CSES 1662 — Subarray Divisibility	https://cses.fi/problemset/task/1662
E	CSES 1746 — Array Description	https://cses.fi/problemset/task/1746
F	AtCoder ABC 151 D — Maze Master	https://atcoder.jp/contests/abc151/tasks/abc151_d
G	Codeforces 1272E — Nearest Opposite Parity	https://codeforces.com/problemset/problem/1272/E
H	Codeforces 1201C — Maximum Median	https://codeforces.com/problemset/problem/1201/C

The order is roughly by difficulty, but as in a real contest, read all problems in the first 20 minutes and solve in the order *you* find easiest.

Stop here until you have sat the contest.

---

5. Debrief

> **Solutions are not published here.** The coach holds the solutions and releases them only after the contest has been run.

Neither the solution ideas nor the reference implementations are included in these notes. Use the debrief time for your own log instead: for each problem, compare when you opened it, when you had an idea, and why each submission was rejected. Section 6 turns that log into a plan.

6. Upsolving and planning

After the contest:

1. **Upsolve every problem within a week.** Once the coach releases a solution, read the idea, close it and write the code yourself.
2. **Analyse your log.** Where did time go — understanding, finding the idea, coding, debugging? Each needs a different fix: more reading practice, more problems per topic, more template code, or a debugging checklist.
3. **Keep a mistake log** of every WA and its cause. After a month you will see that a handful of bug types account for most of your losses.
4. **Re-solve from scratch** after three days. Being able to reproduce a solution cold is the test of whether you learned it.
5. **Plan the next weeks** from your green/yellow/red map. Spend extra practice on red topics that the second half depends on: recursion/DFS (for trees), DP (for every later DP week), binary search (everywhere).

7. Common mistakes (from typical mock contests)

- **Misreading.** "Distinct positions", "1-indexed output", "print any", "modulo 10^9+7" — reread before submitting.
- **Overflow.** If a value can exceed about $2 \cdot 10^9$, use `long long`; check every running sum and every bound you compute, not just the final answer.
- **Negative modulo.** In C++ `%` keeps the sign of the dividend; normalise with `((x % n) + n) % n` whenever inputs can be negative.
- **Wrong binary search invariant.** Decide what `lo` and `hi` mean (e.g. "`lo` infeasible, `hi` feasible") and keep it; then the loop `while (hi - lo > 1)` cannot go wrong.
- **Quadratic hidden in a loop.** Re-running a linear-time routine once per element; erasing from the front of a `vector` in a loop.
- **Not resetting state** between test cases (A has many tests; your later problems may too).
- **Panic-switching.** Abandoning a nearly correct solution after one WA instead of checking the edge cases systematically.

- **Not reading all problems.** An easy problem late in the set is often never opened; read every statement in the first 20 minutes.

8. Practice problems (review set)

- CSES 1090 — Ferris Wheel — https://cses.fi/problemset/task/1090
- CSES 1085 — Array Division — https://cses.fi/problemset/task/1085
- CSES 1641 — Sum of Three Values — https://cses.fi/problemset/task/1641
- CSES 1661 — Subarray Sums II — https://cses.fi/problemset/task/1661
- CSES 1623 — Apple Division — https://cses.fi/problemset/task/1623
- CSES 1095 — Exponentiation — https://cses.fi/problemset/task/1095
- CSES 1645 — Nearest Smaller Values — https://cses.fi/problemset/task/1645
- CSES 1164 — Room Allocation — https://cses.fi/problemset/task/1164
- CSES 1634 — Minimizing Coins — https://cses.fi/problemset/task/1634
- CSES 1194 — Monsters — https://cses.fi/problemset/task/1194
- CSES 1195 — Flight Discount — https://cses.fi/problemset/task/1195
- A virtual participation in any recent Codeforces Div. 3 round (https://codeforces.com/contests), followed by upsolving.

9. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 2 (Time complexity) — the estimation table in more detail.
- George Pólya, *How to Solve It* — the original "understand, plan, carry out, look back" method.
- USACO Guide, "Contest Strategy" and "Debugging Checklist": https://usaco.guide/general/
- Looking ahead: Week 35 covers contest strategy, stress testing and team play in depth; Week 36 is the final review.